

C and Software Tools, Exam 2 Study Guide

October 30, 2024

Exam Structure

- You will have the entire 75 minute meeting time to complete the exam. Keep in mind that the sample exam was for a 50-minute section, so your exam may be a little longer.
- The exam structure will be similar to exam 1:
 - There will be some short-answer questions where you get to answer questions, solve small problems, write short programs or program fragments, and make judgments about code you're given. These questions will represent 45 to 70 percent of the total points on the test.
 - There may be some fill-in-the-blank questions, where you will get to provide bits of C syntax and demonstrate your knowledge of relevant terms. If we have any questions like this on the exam, they'll be worth around 10 to 15 percent of the total exam score.
 - There will be some multiple choice questions, where you will get to choose the best answer from a set of alternatives. These will let you demonstrate knowledge of the programming language, terms and related concepts. They will be worth a total of 15 to 30 percent of the points.
- Materials
 - You may bring with you one 3×5 inch note card with any notes you would like to have for the exam. You may write notes on both sides of the card. Note cards may not be shared between students, they must be handwritten, and will be collected when you turn in your exam.
 - If you have a note card, your name must be clearly printed on it.
 - If your note card is too large, I'll have to take it away. If you start taking the exam with a note card that's too large, your score on the exam will be penalized.
 - You may not use your textbook or other notes during the exam.
 - You may not use any electronic devices (e.g., calculators, cell phones) during the exam.
- If you must leave the classroom for a bathroom break, you must notify and leave your cell phone with the teaching staff. Only one student at a time may leave the room at a time. Late in the exam period, students may not be permitted to take a bathroom break.
- The point total on the exam may be more than 100. However, it will be treated as a 100-point exam in calculating your class average. If you earn more than 100 points, you've earned yourself some extra credit points.

Exam Topics

For the exam, you should be able to do the following. If you are expected to be able to use a particular programming construct (e.g., a function pointer), then you need to be able to produce code using that construct (e.g., code that calls a function given a pointer to the function), explain the construct (e.g., report what happens when you take the address of a function or why you would want to use a function pointer) and make judgments about code using that type of construct (e.g., determine what will happen when a particular function pointer is called).

- Size of fundamental types on the common platform (e.g., int, short, double, pointers).
- Use the size of fundamental types on the common platform to determine the size of arrays and the size and padding in structs.
- Use the C99 syntax to create anonymous arrays containing particular sequences of values (e.g., `(int []) { 1, 7, 15 }`).
- Files and I/O
 - Open files for reading and writing. Write formatted output to a file using `fprintf()`, and read formatted input using `fscanf()`.
 - Use parsing features of `scanf()` / `fscanf()`, including limits on string length, character classes, matching literal characters, skipping spaces and ignoring fields.
 - Read and write individual characters / bytes using `fgetc()` and `fputc()`.
 - Use the pattern matching syntax supported by `scanf()` and `fscanf()` to match inputs of particular formats (e.g., sequences of capital letters, a word of at most 20 characters).
 - Use `rewind()`, `fseek()` and `ftell()`.
 - Use `feof()` to handle end-of-file detection.
 - Use `fflush()` and `ungetc()` and explain what buffering is and why it's done in I/O.
- Use `sscanf()`, `sprintf()` and `snprintf()` to parse input from a string and send output to a string.
- Pointers
 - Demonstrate and explain valid and invalid uses of pointer types as function return values.
 - Read and create complex types including pointers, function pointers, const pointers, and pointer types for single- and multi-dimensional arrays and structs.
 - Explain how arrays are stored and the relationship between pointer and arrays.
 - Use post/pre increment/decrement in combination with the pointers and the dereference operator.
 - Describe the behavior of pointer arithmetic, and use it to index into arrays and to compare and measure the difference between two pointers in the same array.
 - Describe and use the relationship between the `a[ i ]` syntax and the `*( a + i )` syntax.
 - Use and compare pointer syntax and array syntax for traversing and manipulating arrays.
 - Describe and use the relationship between the `int a[]` syntax and the `int *a` syntax for parameter passing.
 - Describe differences between a pointer and an array. Use code that includes arrays and pointers.
 - Use literal strings, describe how they are allocated and how they can and can't be used.
 - Describe the representation of strings in C, and write code to access, copy and modify portions of string.
 - Use and compare two-dimensional arrays (arrays of arrays) and arrays of pointers as alternatives for creating two-dimensional organizations of characters, strings and other data types. Identify and explain sources of overhead for each technique.
 - Allocate and use representations consisting of single- and multi-dimensional arrays and structs, stored either by value or by address (e.g., create an array of structs vs. an array of pointers to structs).
 - Extract and use pointers to elements, rows and offsets within a multi-dimensional array or arrays of pointers.
 - Use command-line arguments.
 - Use void pointers and explain their purpose.

- Declare and use function pointers to copy the address of a function and to call a function given an address. Explain how function pointers are useful.
 - Use `atexit()` to register a function for execution at successful program termination.
 - Use `qsort()` to sort arrays of arbitrary types using an arbitrary comparison function.
- Structs
 - Define structs containing fields of various types, including arrays, pointers and other structs. Create instances of structs immediately after the struct definition or elsewhere in code. Create struct instances statically, as auto variables and dynamically using `malloc`.
 - Describe and use structure names (i.e., structs that haven't been typedef-ed to a type name).
 - Use structs in assignments, by value and by address. Pass and return structs from functions by value and by address. Use a struct as a way of returning multiple values from a function.
 - Access struct fields given a struct value or a pointer to a struct. Use the `.` and `->` operators in combination with other operators and make judgments about their precedence.
 - Use both types of initialization syntax (C99 syntax with field names and the older syntax without field names) in a variable declaration. Use the C99 syntax to create anonymous structure values outside a variable declaration (e.g., `(struct Pair) { 5, 15 }`). Make judgments about the contents of structures after they are created or operated on.
 - Describe how struct instances are mapped to memory and why the compiler may include padding inside a structure and the related topic of alignment requirements.
 - Use nesting of structures, arrays and pointers. Initialize variables with these types and use the `.`, `->` and `[ ]` operators to access parts of these kinds of values.
 - Use bitfields in a struct and describe their advantages and disadvantages compared to using bitwise operators directly.
 - Dynamic Memory Allocation and Data Structures
 - Describe the lifetime of heap-allocated storage, how it is used in a program and why. Compare and contrast heap memory with stack and static storage.
 - Use `malloc()` to allocate space for fundamental types, structs and arrays, and use `free()` to return this memory to the heap.
 - Describe and define memory leaks and dangling pointers and find examples of these errors in sample source code.
 - Describe and use `calloc()` and `realloc()`. Explain what they are for, and how they differ from `malloc()`.
 - Use `malloc()`, `realloc()` and `free()` to implement a resizable array.
 - Use `memcpy()` and `memmove()` and describe the difference between them.
 - Describe the tradeoffs between arrays and linked structures as alternatives for data structure implementation.
 - Implement a linked list with dynamically allocated nodes. Write code to traverse a linked list, to insert and remove nodes from the start and end of the list, and to free memory for the list.
 - Describe and implement alternatives for implementing a function to modify a linked list (e.g., returning an updated head pointer, taking a head pointer passed by address).
 - Use pointer-to-pointer to simplify code to insert or remove a node from a linked list.
 - Bitwise operations
 - Use bitwise and, or, exclusive or and complement operators.

- Use left and right shift operators, describe the difference between logical and arithmetic right shift, and use them for special cases of multiplication and division.
- Use bitwise operators to set, clear and test bits in a larger value. Use bitwise and other operators to access and manipulate bit fields in a larger value.
- Read and write octal and hexadecimal values using `scanf()` and `printf()`.
- Use `typedef` to create short names for more complex types and to simplify type definitions for arrays of arrays and for types using function pointers. Describe, in general, why `typedef` can be helpful.
- Appropriately use `assert()`, describe what it's for, and how it's supported in C. Explain how to disable assertions during compilation.
- Tools
 - Use gcc options to limit error messages and warnings in the output.
 - Use gcc instrumentation options to enable detection of runtime errors (buffer overflow, null pointer dereference, use after free, etc).
 - Compile C programs to measure code coverage, collect and interpret code coverage results using `gcov`.
 - Use `valgrind` to run a program, looking for memory leaks, use of uninitialized memory, access to invalid addresses and file leaks.
 - Interpret `valgrind` error messages.
 - Use the `sgcheck` tool in `valgrind` and describe what it does.
 - Describe `cppcheck`. Show how to run it and describe the difference between static and dynamic analysis.
 - Use and describe shell syntax for pipes, for redirecting streams (standard input, output and error) and paginating output.
- The C Standard Library and Library Functions
 - Briefly describe the headers, `limits.h`, `stdbool.h`, `assert.h`, `string.h`, `ctype.h`, `stdio.h`, `stdlib.h`, `errno.h`, `math.h` and `stdarg.h`.
 - Open a file in binary mode and read/write the file using `fread()` and `fwrite()`.
 - Describe LSB-first and MSB-first byte order and write code to reverse byte order in ints and shorts.
 - Describe the `rand()` and `srand()` functions and how `rand()` can be used to generate pseudo-random values in desired ranges.
 - Describe the use of the `errno` variable and the `perror()` function.
 - Use functions `sin()`, `cos()`, `tan()`, `asin()`, `acos()`, `atan()`, `atan2()`, `pow()`, `sqrt()`, `round()`, `floor()` and `ceil()` from `math.h`, including linking with the math library. Use the variants of these functions for other types.
- Floating Point Numbers
 - Describe differences in the representation of the three floating point types (float, double and long double) and how these relate to the capacity of each type.
 - Describe and use the IEEE single (float) and double precision format. Convert simple values between decimal and the IEEE format.
 - Describe overflow and underflow in floating point calculations and identify cases where it will occur.
 - Describe what happens when converting between integer and floating point types.